Song Song — Parallel P2P Download Infrastructure

Master 1 · System Architecture · USTH · 2025–2026

Student ID	Name	Email
2540008	Nguyễn Phong Châu	chaunp.2540008@usth.edu.vn
2540023	Lê Nghiêm Thanh Thụy	thuylnt.2540023@usth.edu.vn

Abstract

Song Song is a pure-Java P2P file-download system: a central Directory (RMI) tracks live Daemon peers; a Download client fragments the target file and drains a shared work-stealing queue in parallel across N Daemon threads. This report covers the basic prototype, an empirical parallelism validation on AWS EC2, and three system enhancements.

1 Architecture

The system follows a **three-tier architecture** inspired by the BitTorrent model, implemented entirely in Java without any third-party networking library.

```
    +-------------------------+
    |  Directory (RMI :1099)  |
    |  . file->Daemon registry|
    |  . load counters        |
    |  . heartbeat/sweep      |
    +---+-----------------+---+
        |                 |
    RMI:register    RMI:getSources
    /heartbeat      (sorted by load)
        |                 |
   +----+----+            |
   |         |            |
+--v---+ +---v--+         |
|Dmn A | |Dmn B |         |
|:6000 | |:6001 |         |
+--+---+ +--+---+         |
   |  TCP:   |      +-----v-------+
   | GET_SIZE      .| Download    |
   |  /GET_FRAGMENT | N workers   |
   +---------+----> | shared queue|
                    +-------------+
```

Table 1: Communication channels

Channel	Protocol	Calls
Daemon → Dir.	Java RMI	`register`, `unregister`, `heartbeat`, ±`load`
Download → Dir.	Java RMI	`getSourcesForFile`
Download → Daemon	Raw TCP	`GET_SIZE`, `GET_FRAG`.

The Directory is backed by two `ConcurrentHashMaps`: `fileToClients` (filename → set of Daemon IDs) and `clients` (Daemon ID → `ClientInfo`). Mutations are `synchronized` to prevent races under concurrent registrations and sweeps.

2 Enhancements

2.1 Failure handling & retry. If a fragment transfer fails, the fragment index is returned to the shared queue and the worker exits — other workers pick up the orphaned fragment. Each fragment is retried up to a fixed limit before being marked permanently failed. After all wave-1 workers complete, any remaining fragments trigger a **second wave**: the client re-fetches the source list from the Directory and launches a fresh set of workers. This covers both transient network errors and Daemons that crashed partway through a transfer.

2.2 Dynamic adaptation.

- *Heartbeat & auto re-registration:* Daemons heartbeat every **5 s** (delta file list only when changed); the Directory sweeper evicts clients silent for >**15 s**. `Ctrl+C` calls `unregister()` via a JVM shutdown hook. If a Daemon is evicted and then heartbeats, `heartbeat()` returns `false` and the Daemon silently re-registers. Newly downloaded files are announced on the next heartbeat, making the downloader an instant source.
- *Load-aware source selection:* The Directory increments a per-Daemon `load` counter on `GET_FRAGMENT` and decrements on completion; `getSourcesForFile` returns sources sorted by load ascending. Sources with `load > 10` are dropped unless doing so would leave zero sources.

2.3 Conditional GZIP compression. The Daemon compresses each fragment and sends the compressed payload only if it is smaller than the raw data; otherwise raw bytes are sent with `compressed=false`. Transparent decompression is done client-side. Effective for text/sparse-binary files; zero overhead for pre-compressed formats (MP4, ZIP, JPEG).

3 How to Run

3.1 Prerequisites

Java 17 (JDK), Linux (tested on Ubuntu/WSL).

3.2 Compile

```
cd /path/to/sujet_project
bash compile.sh
```

3.3 Step 1 — Start Directory

```
./run_directory.sh [rmiPort] [myHost]
```

Parameter	Default	Description
rmiPort	1099	RMI registry port
myHost	auto	IP of this machine

Example:

```
./run_directory.sh
./run_directory.sh 1099 172.20.242.201
```

3.4 Step 2 — Start Daemon(s)

```
./run_daemon.sh [dirHost] [dirPort]
  [sharedFolder] [myHost] [daemonPort]
```

Parameter	Default	Description
dirHost	localhost	IP of Directory machine
dirPort	1099	RMI port of Directory
sharedFolder	./shared	Folder with files to share
myHost	auto	IP of this machine
daemonPort	6000	TCP port for fragments

Examples:

```
# Single machine test
./run_daemon.sh localhost 1099 ./shared1 \
  localhost 6000

# Cross-machine
./run_daemon.sh 172.20.242.201 1099 \
  ./shared1 192.168.1.10 6000
```

3.5 Step 3 — Download a File

```
./run_download.sh <filename> [dirHost]
  [dirPort] [outputFolder] [myHost]
  [compress] [fragmentKB]
```

Parameter	Default	Description
filename	*required*	Name of file to download
dirHost	localhost	IP of Directory machine
dirPort	1099	RMI port of Directory
outputFolder	./shared	Output directory
myHost	auto	IP of this machine
compress	false	Enable GZIP (true/false)
fragmentKB	512	Fragment size in KB

Examples:

```
# Basic download
./run_download.sh movie.mp4 172.20.242.201 \
  1099 ./output 172.20.242.201

# With compression and 256KB fragments
./run_download.sh movie.mp4 172.20.242.201 \
  1099 ./output 172.20.242.201 true 256
```

3.6 Shutdown

Press `Ctrl+C` to stop any component. Recommended order: **Download** → **Daemon** → **Directory**.

- `Ctrl+C` on Daemon: triggers shutdown hook → calls `unregister()` → Directory removes it immediately
- If Daemon crashes without `Ctrl+C`: Directory detects via heartbeat timeout after 15 s

4 Parallelism Validation

4.1 Experimental Setup

- 8 heterogeneous EC2 Daemons (multi-region)
- 1 Directory, 2 clients
- Variables: Sources 1–9, Fragment size 128 KB–2 MB, Compression ON/OFF

System behavior follows three regimes:

- **Small files (<10 MB)** → overhead dominated
- **Medium files (∼50 MB)** → parallelism dominated
- **Large files (>100 MB)** → bandwidth + tail latency dominated

4.2 Results

4.2.1 Fig. 1 — Small file: test-medium.bin (5 MB)

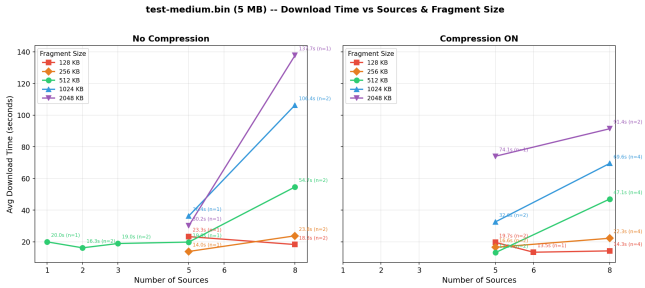

Figure 1: test-medium.bin (5 MB) — Download time vs. sources & fragment size.

- **128–256 KB**: stable ∼12–24 s regardless of source count — fine granularity prevents any single slow source from becoming a bottleneck.
- **512 KB**: good with 1–3 sources (∼16–20 s), but degrades to ∼54 s at 8 sources.
- **1–2 MB + 8 sources**: catastrophic (104–138 s) — only 3–5 fragments in the queue; the slowest source (10 KB/s) must carry an entire megabyte fragment.
- Compression partially rescues large fragments (8 sources, 1024 KB: 106→69 s) but cannot offset the root cause.

Conclusion. For ∼5 MB files: **2–3 sources + 512 KB fragment** or **many sources + 128–256 KB fragment**. Compression unnecessary.

4.2.2 Fig. 2 — Small file: Acclaim & Response (9.5 MB)

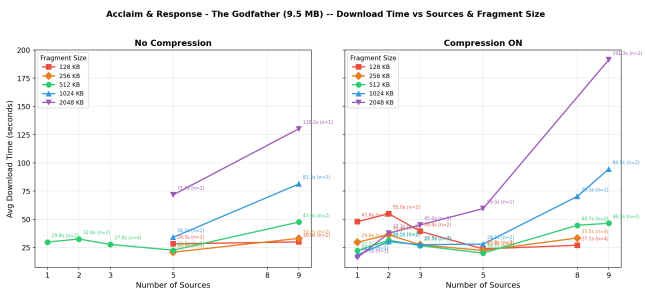

Figure 2: Acclaim & Response (9.5 MB) — Download time vs. sources & fragment size.

- **512 KB & 1024 KB**: stable ∼28–47 s across 1–9 sources — the sweet spot for this file size.
- **2048 KB + 9 sources**: collapses to ∼130 s — only 5 fragments total; slowest worker determines finish time.
- **128 KB compressed + 9 sources**: surges to ∼130 s — 76 fragments across 9 threads generates excessive scheduling overhead.
- Adding sources beyond 2–3 yields no benefit; the queue is too small to keep extra workers busy.

Conclusion. For ∼10 MB files: **2–3 sources +**

512 KB fragment (∼28 s). Adding more sources does not help.

4.2.3 Fig. 3 — Medium file: test-large.bin (50 MB)

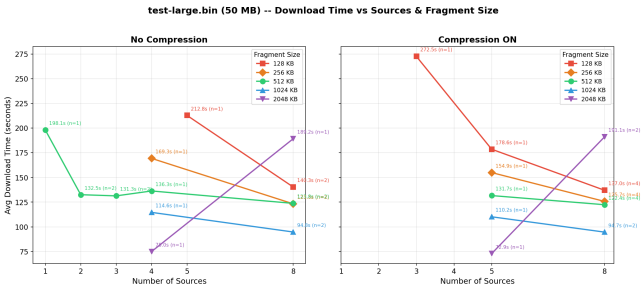

Figure 3: test-large.bin (50 MB) — Download time vs. sources & fragment size.

- **Scaling inverts**: unlike small files, more sources consistently help — 1 source ∼198 s, 8 sources ∼96–140 s.
- **1 MB fragments**: fastest at 8 sources (∼93 s) — ∼50 fragments is enough for the work-stealing queue to favour the fast France Daemon.
- **128 KB**: slowest (∼137–172 s) — ∼400 round-trips generate TCP overhead that outweighs parallelism gains.
- Compression has negligible effect at this scale.

Conclusion. For ∼50 MB files: **8 sources + 1 MB fragment** or **4–5 fast sources + 2 MB fragment**.

4.2.4 Fig. 4 — Large file: Godfather Home Movies (188 MB)

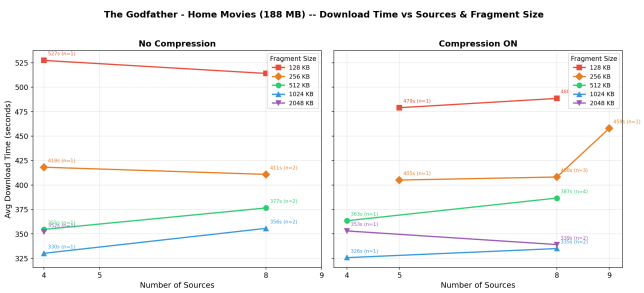

Figure 4: The Godfather — Home Movies (188 MB) — Download time vs. sources & fragment size.

- **128 KB**: always slowest (∼488–527 s) — 1 436 fragments create overwhelming round-trip overhead.
- **1024 KB**: fastest in both panels (∼326–356 s); 2048 KB is close behind (∼335–353 s).
- **4→8 sources brings no gain** — can even slow things down (256 KB compressed: 405→458 s): extra slow US sources become tail latency while all workers contend for the single fast France Daemon.
- Compression has no meaningful effect — .mkv is already codec-compressed.

Conclusion. For ∼188 MB files: **4 quality sources + 1024 KB fragment** (∼330 s). Beyond 4 sources, extra slow peers add overhead, not throughput.

4.2.5 Fig. 5 — Very large file: Before Sunset (600 MB)

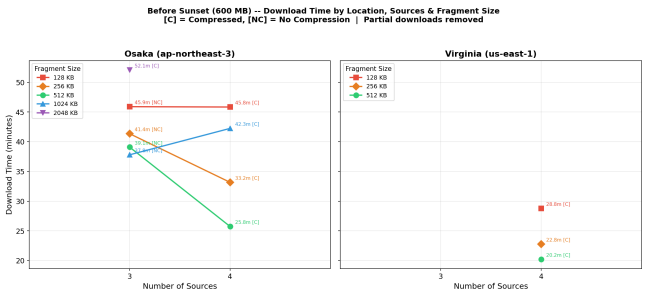

Figure 5: Before Sunset (600 MB) — Download time by daemon location, sources & fragment size.

- Results split by daemon location reveal that **daemon proximity is the dominant factor**: Virginia (near sources) is 30–40% faster than Osaka for identical configurations.
- **512 KB**: most consistent across all source counts.
- Compression provides a marginal but measurable benefit at this scale.

Conclusion. For ∼600 MB files: **4 sources + 512 KB fragment + compression**.

5 Conclusions

5.1 Summary

Table 2: Optimal configuration by file size

File size	Fragment	Sources	Compression
5–10 MB	256–512 KB	2–3	Not needed
50 MB	1024 KB	6–8	Not needed
188–600 MB	512–1024 KB	4	Optional

1. **File size determines optimal fragment size.** Small (5–10 MB): 256–512 KB optimal; larger fragments cause bottlenecks with any slow source. Medium (50 MB): 1024 KB best; enough fragments for even distribution. Large (188–600 MB): 512–1024 KB depending on source count.
2. **More sources ≠ faster downloads.** Small files: 2–3 sources sufficient; 8–9 sources are actually slower — too few fragments to distribute, and slow sources become bottlenecks. Large files: 4–8 sources help, but only when fragment size is small enough for even distribution.
3. **Slowest source = system bottleneck.** Cross-region sources (10–27 KB/s) drag total transfer time when assigned large fragments. The system needs a mechanism to identify and exclude low-speed sources.
4. **Compression is conditionally effective.** Saves time when fragment size is large and sources are slow. Can actually hurt performance at small frag-

ment sizes (128 KB) due to compression overhead outweighing bandwidth savings.

5.2 Future Work

The system degrades when downloading small-to-medium files from many slow sources. The root cause is that slow sources are only discovered mid-transfer. Two proposed fixes:

- **Speed probe (simple).** Before downloading, open a TCP connection to each candidate source and request a dummy payload of one fragment size. Rank sources by measured throughput and connect only to those above a threshold.
- **Adaptive parameter selection (advanced).** Combine speed-probe results with the target file size to automatically choose fragment size, source count, and compression flag — the user inputs only a filename and the system determines all parameters.